

Read - Write

Read Miss

load into cache

early restart /
load through to CPU



Write

back

through

Write Through on Hit

updates memory and cache ignored
purpose of cache write buffer.
get around using

valid bit

Write back on hit

update cache only.
set dirty bit

valid + dirty bit

to replace:

valid	dirty	tag	data
1	1	-	A=11

3

2

Write back on miss

write allocate
bring into cache
update

write no allocate
update directly
useful for data
not going to be used
immediately

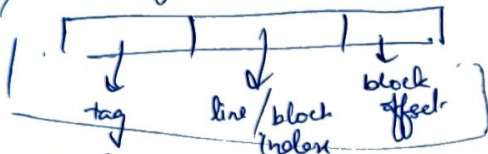
1

Cache Mapping Techniques

Direct mapping

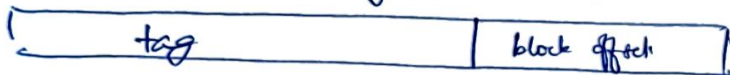
Block no % no of blocks in cache;

• trivial replacement simple but ~~not~~ not feasible



Fully Associative

Anywhere in cache; searching more expensive; replacement of

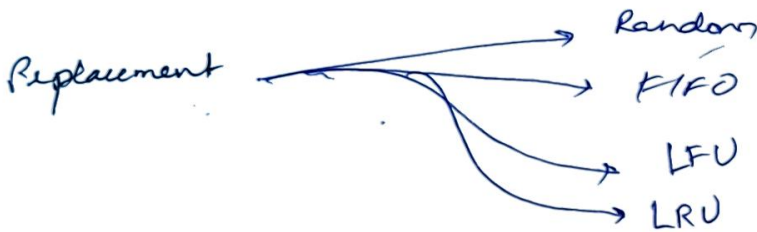
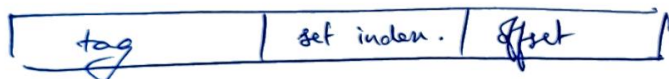


Set Associative

divide into sets

Block no % no of sets

within set; anywhere replacement algo..



Performance

- $AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$
- $CPU \text{ Exec} = IC \times \text{overall CPI} \times \text{cycle time}$
- $\text{Overall CPI} = \text{Base CPI} + \text{CPU stall} + \text{mem stall}$
- $\text{Mem stall} = \text{mem access} \times \text{miss rate} \times \text{miss penalty}$
- Unified data + inst cache: $(1 + \alpha) \text{ mem access}$

Cache Optimisations

miss penalty_{L1}

Multilevel cache

$$\begin{aligned} \text{Avg mem stall} &= \underbrace{\text{miss per instr}}_{L1} \times \text{hit time}_{L2} \\ &\quad + \underbrace{\text{miss per instr}}_{L2} \times \text{miss penalty}_2 \\ &= (\text{AMAT} - \text{hit time}_{L1}) \times \text{avg mem ref/instr}. \end{aligned}$$

Everything in L1 $\rightarrow$ in L2

Prioritising read over write

Write through: Read miss $\rightarrow$ check buffer $\xrightarrow{\text{no conflict}}$ continue

Write back: Dirty block $\rightarrow$ copy to buffer $\rightarrow$ read from mem $\rightarrow$ write later.
read miss $\rightarrow$ check buf